

1. What is a Trigger?

A Salesforce **Apex Trigger** is code that runs **automatically** when something happens to a Salesforce record.

Basic Syntax:

```
trigger TriggerName on ObjectName (event) {  
  
    // code  
  
}
```

Example:

```
trigger CustomerTrigger on Customer__c (before insert) {  
  
    // code  
  
}
```

Flow:

Record operation
↓
Trigger automatically fires
↓
Your Apex code runs

For example, when a new Customer is created:

Create Customer
↓
Trigger fires automatically
↓
Set Membership Status = "Inactive"

2. Trigger Events

There are 7 main trigger events:

Event	When does it run?
before insert	Before a new record is saved
after insert	After a new record is saved
before update	Before an existing record is updated
after update	After an existing record is updated
before delete	Before a record is deleted
after delete	After a record is deleted
after undelete	After a deleted record is restored

Before Trigger:

Use before triggers mainly when you want to change or validate the same record before Salesforce saves it.

Example:

User creates Customer
Name = Vaibhav

↓ before insert

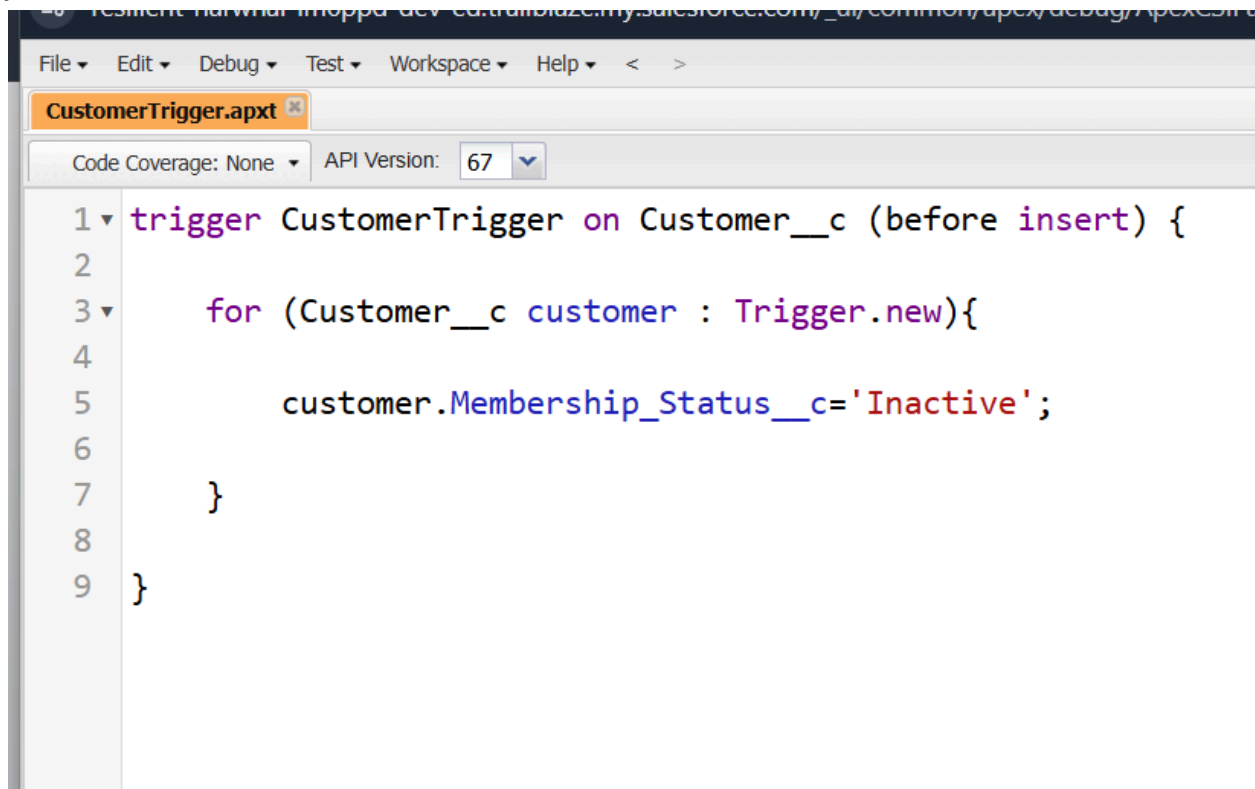
Automatically set:
Membership Status = Inactive

↓

Salesforce saves Customer

Code:

```
trigger CustomerTrigger on Customer__c (before insert) {  
  
    for (Customer__c customer : Trigger.new) {  
  
        customer.Membership_Status__c = 'Inactive';  
  
    }  
}
```



Related	Details
Customer Name anil	Owner vaibhav gidda
Phone 937473900	
Email anil@gmail.com	
Gender Male	
Join Date 02/09/2026	
Membership Status Inactive	
Created By vaibhav gidda, 07/08/2026, 10:25 am	Last Modified By vaibhav gidda, 07/08/2026, 10:25 am

Automatically membership status given Incative.without giving the membership status while iam creating the customer .

Trigger.new:

contains the new/current versions of the records being processed.

```
for (Customer__c customer : Trigger.new) {
//Go through every Customer currently being processed.

}
```

Trigger.new vs Trigger.old

During an update, Salesforce can give us both versions.

Example:

Before edit:

Membership Type = Monthly

After edit:

Membership Type = Yearly

Then conceptually:

Trigger.old



Monthly

Trigger.new



Yearly

Roadmap:**BASIC TRIGGERS**

1. before insert
2. before update
3. after insert
4. before delete

**UNDERSTAND TRIGGER DATA**

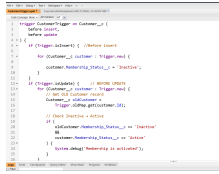
5. Trigger.new / Trigger.old
6. Trigger.newMap / Trigger.oldMap

**ENTERPRISE LEVEL**

7. Bulkification
8. Trigger Handler Pattern

Topic	Simple meaning
before insert	Do something before creating a record
before update	Do something before updating a record
after insert	Do something after record is created
before delete	Check/do something before deletion
Trigger.new w	Gives new/current records
Trigger.old d	Gives previous records
newMap / oldMap	Access new/old records quickly using their IDs
Bulkification	Make trigger work safely for 1 or many records
Trigger Handler	Move large logic out of Trigger into a separate Apex class

Before Update:



```
trigger CustomerTrigger on Customer__c (  
    before insert,  
    before update  
) {
```

```
    // BEFORE INSERT
```

```
    for (Customer__c customer : Trigger.new) {  
  
        customer.Membership_Status__c =  
'Inactive';  
    }  
}
```

```
    // BEFORE UPDATE  
    if (Trigger.isUpdate) {
```

```
        for (Customer__c customer : Trigger.new) {  
  
            // Get OLD Customer record  
            Customer__c oldCustomer =  
                Trigger.oldMap.get(customer.Id);  
  
            // Check Inactive → Active  
            if (  
                oldCustomer.Membership_Status__c  
                == 'Inactive'  
                &&  
                customer.Membership_Status__c ==  
                'Active'  
            ) {  
  
                System.debug('Membership is  
activated');
```

```
}  
}  
}  
}
```

File Edit Debug Test Workspace Help < >		
CustomerTrigger.apxt Log executeAnonymous @8/7/2026, 11:43:07 AM		
Execution Log		
Timestamp	Event	Details
11:43:07:003	USER_DEBUG	[32] DEBUG Membership is activated

After Insert:

`after insert` runs **after** a new record has been successfully saved.

Difference:

BEFORE INSERT

New record



Trigger



Change/validate record



SAVE

AFTER INSERT

New record



SAVE



Trigger



Perform another action

// AFTER INSERT

```
if (Trigger.isAfter && Trigger.isInsert) {
```

```
    for (Customer__c customer : Trigger.new) {
```

```
        System.debug(
```

```
            'Customer created successfully. ID = ' + customer.Id
```

```
        );
```

```
    }
```

```
}
```

CustomerTrigger.apxtLog executeAnonymous @8/7/2026, 11:43:07 AMCustomerstrigger.apxtLog executeAnonymous @8/7/2026, 12:16:16 PM

Code Coverage: NoneAPI Version: 67

```
11 }
12 }
13 if (Trigger.isBefore && Trigger.isUpdate) { // BEFORE UPDATE
14     for (Customer__c customer : Trigger.new) {
15         // Get OLD Customer record
16         Customer__c oldCustomer =
17             Trigger.oldMap.get(customer.Id);
18
19         // Check Inactive → Active
20         if (
21             oldCustomer.Membership_Status__c == 'Inactive'
22             &&
23             customer.Membership_Status__c == 'Active'
24         ) {
25             System.debug('Membership is activated');
26         }
27     }
28 }
29
30 //after insert
31 if (Trigger.isAfter && Trigger.isInsert){
32     for(Customer__c customer : Trigger.new){
33         System.debug( 'Customer created successfully. ID = ' + customer.Id);
34     }
35 }
36 }
```

LogsTestsCheckpointsQuery EditorView StateProgressProblems

Filter

FileEditDebugTestWorkspaceHelp<>

CustomerTrigger.apxtLog executeAnonymous @8/7/2026, 11:43:07 AMCustomerstrigger.apxt

Execution Log

Timestamp	Event	Details
12:16:16:001	USER_DEBUG	[3] DEBUG insertion operation
12:16:16:065	USER_DEBUG	[33] DEBUG Customer created successfully. ID = a00dM00003hzZWQQA2

Why `Trigger.isAfter` && `Trigger.isInsert`?

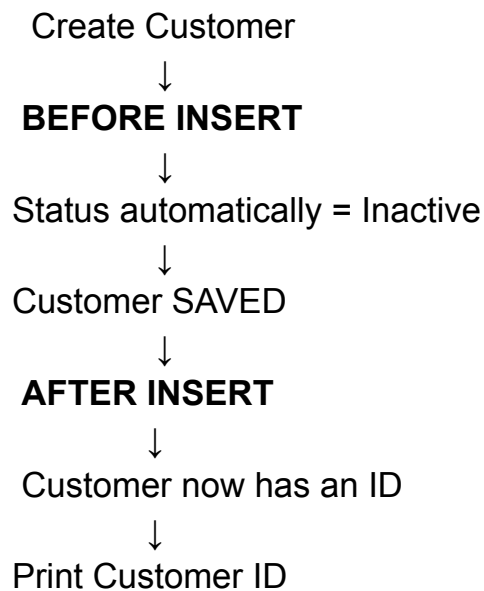
`Trigger.isAfter`

= Are we running AFTER save?

`Trigger.isInsert`

= Is this an INSERT operation?

How it Executes:

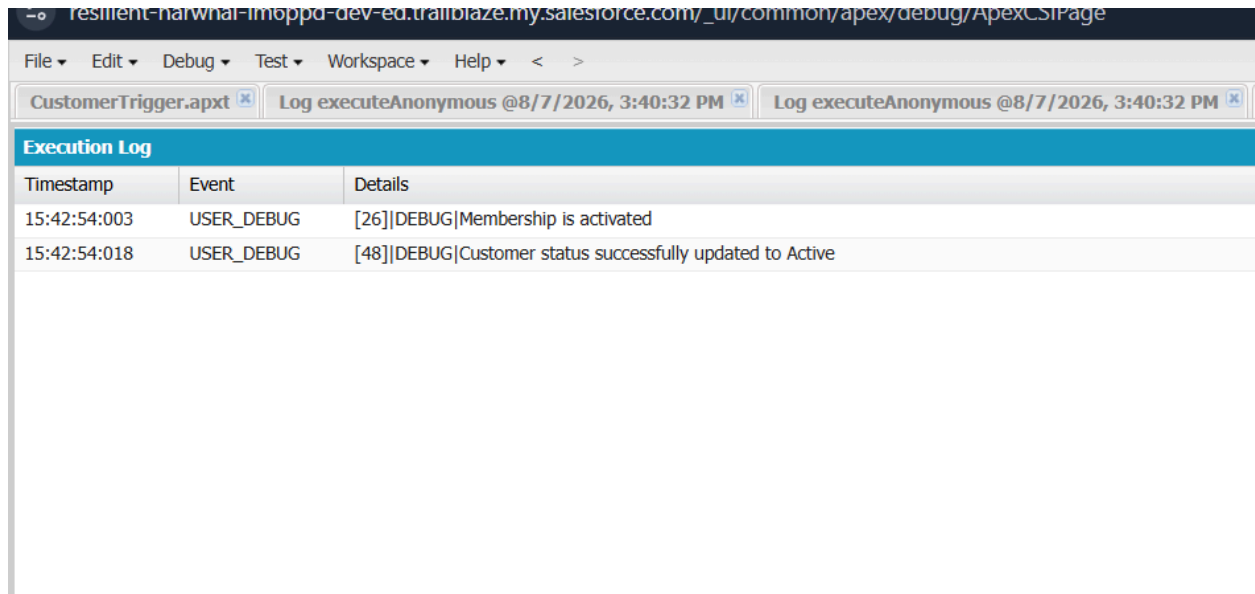


After Update:

```
// AFTER UPDATE
if (Trigger.isAfter && Trigger.isUpdate) {
  for (Customer__c customer : Trigger.new) {
    Customer__c oldCustomer =
      Trigger.oldMap.get(customer.Id);

    if (oldCustomer.Membership_Status__c == 'Inactive'
      && customer.Membership_Status__c == 'Active') {
      System.debug('Customer status successfully updated to Active');
    }
  }
}
```

}



The screenshot shows the Salesforce Apex debug console. The browser address bar displays the URL: `resilient-harwhal-fmoppd-dev-ed.trailblaze.my.salesforce.com/_ui/common/apex/debug/ApexCSIPage`. The console has tabs for `CustomerTrigger.apxt`, `Log executeAnonymous @8/7/2026, 3:40:32 PM`, and another `Log executeAnonymous @8/7/2026, 3:40:32 PM`. Below the tabs is the **Execution Log** table.

Timestamp	Event	Details
15:42:54:003	USER_DEBUG	[26] DEBUG Membership is activated
15:42:54:018	USER_DEBUG	[48] DEBUG Customer status successfully updated to Active

before delete :

It runs before Salesforce deletes a record.

Example requirement:

Don't allow an **Active Customer** to be deleted.

Flow:

User clicks Delete



BEFORE DELETE fires



Check Membership Status



Active?

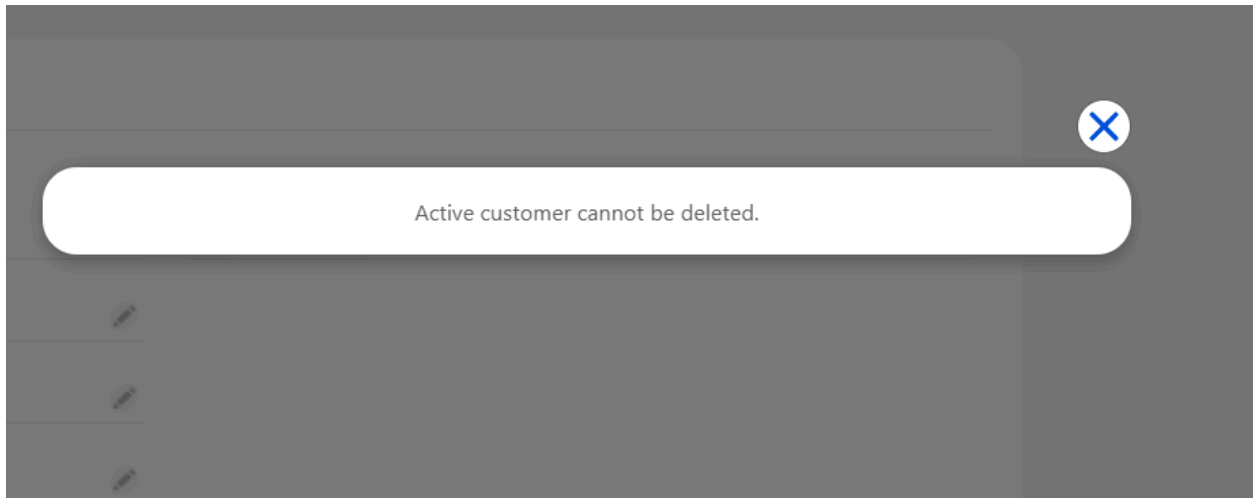


YES → Stop deletion ❌

```

if (trigger.isbefore && trigger.isdelete){
    for (Customer__c customer: trigger.old){
        if (customer.Membership_Status__c=='active'){
            customer.addError('Active customer cannot be deleted.');
        }
    }
}
}

```



After Deletion:

It runs after Salesforce successfully deletes a record.

// AFTER DELETE

```

if (Trigger.isAfter && Trigger.isDelete) {

    for (Customer__c customer : Trigger.old) {

        System.debug(
            'Customer deleted successfully: ' + customer.Name
        );
    }
}

```

```
58 ▾ if (trigger.isAfter && trigger.isDelete){  
59 ▾   for(Customer__c customer : trigger.old){  
60     System.debug('customer deleted sucessfully'+customer.Name);  
61   }  
62 }  
63 }  
64
```

ogs Tests Checkpoints Query Editor View State Progress Problems

Inactive Customer



Click Delete



BEFORE DELETE



Allowed 



Customer deleted



AFTER DELETE



Debug:

"Customer deleted successfully: Krishna"

Difference:

BEFORE DELETE

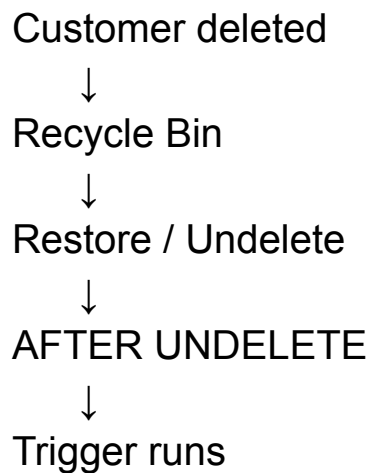
- record not deleted yet
- can STOP deletion using `addError()`

AFTER DELETE

- record already deleted
- perform an action after deletion

What is **after undelete**?

When a deleted record is restored from the **Recycle Bin**, Salesforce runs **after undelete**.



```
// AFTER UNDELETE
if (Trigger.isAfter && Trigger.isUndelete) {

    for (Customer__c customer : Trigger.new) {

        System.debug(
            'Customer restored: ' + customer.Name
        );
    }
}
```

Go to Applauncher:

The screenshot shows a 'Recycle Bin' interface with the title 'My Recycle Bin'. It contains two records in a table. The table has columns for 'Record Name', 'Type', 'Deleted By', and 'Deleted Date'. The first record is 'anil', a 'Customer' deleted by 'vaibhav gidda' on '07/08/2026, 4:05 pm'. The second record is 'vibe', also a 'Customer' deleted by 'vaibhav gidda' on '30/07/2026, 5:34 pm'. Above the table, there are buttons for 'Restore', 'Delete', and 'Empty Org Recycle Bin'. Below the table, there are icons for settings, refresh, and other actions.

	Record Name	Type	Deleted By	Deleted Date
1	anil	Customer	vaibhav gidda	07/08/2026, 4:05 pm
2	vibe	Customer	vaibhav gidda	30/07/2026, 5:34 pm

Why Trigger.new?

For delete:

Trigger.old → deleted record

For undelete:

Trigger.new → restored record

Flow:

anil deleted



Recycle Bin



Restore



after undelete



Trigger.new



Customer restored successfully: anil

